BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Đặng Ngọc Như Ý
MSSV / Lớp	3123410438 / DTN1241

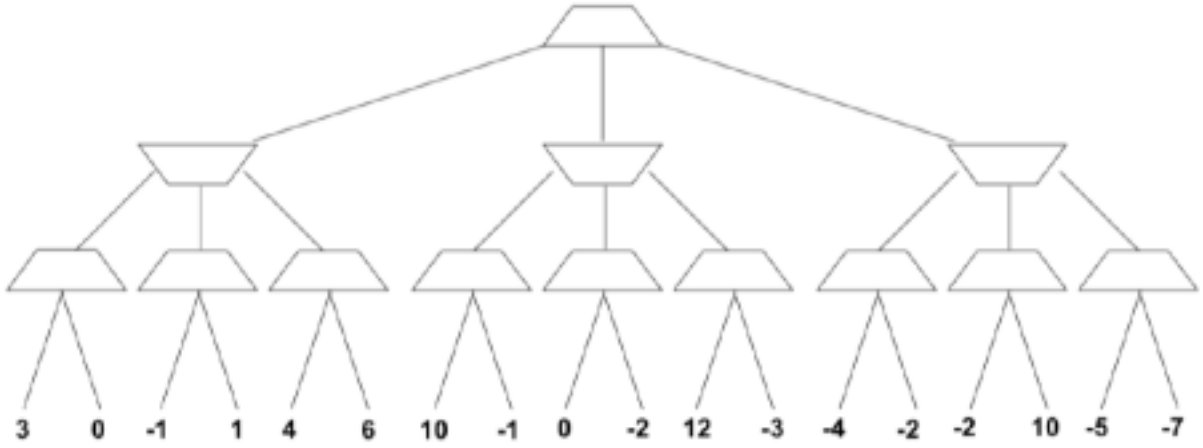
Mục tiêu học tập

• Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa. •
Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.
• Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP. • Giải thích
tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi. • Phân tích
tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

max(3, 0) = 3; max(-1, 1) = 1; max(4, 6) = 6; max(10, -1) = 10; max(0, -2) = 0; max(12, -3) = 12; max(-4, -2) = -2; max(-2, 10) = 10; max(-5, -7) = -5

min_{trai} = min(3, 1, 6) = 1; min_{giua} = min(10, 0, 12) = 0; min_{phai} = min(-2, 10, -5) = -5

V = max(1, 0, -5) = 1

b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

Nhánh trái: 1

Nhánh giữa: 0

Nhánh phải: -5

Vì 1 là giá trị lớn nhất nên MAX chọn nhánh trái, V = 1

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

Ban đầu tại gốc MAX: $\alpha = -\infty$, $\beta = +\infty$. Ở nút MIN trái, nút MAX thứ nhất trả về 3 nên $\beta = 3$; nút MAX thứ hai trả về 1 nên β giảm còn 1. Sang nút MAX thứ ba, lá đầu tiên là 4; tại đây $\alpha = 4 \geq \beta = 1$ nên cắt lá 6. Nút MIN trái trả về 1 và cập nhật α ở gốc thành 1.

Ở nút MIN giữa, nút MAX đầu tiên trả về 10. Nút MAX tiếp theo trả về 0, làm β của MIN bằng 0. Vì $\beta = 0 \leq \alpha = 1$, cắt nút MAX còn lại cùng hai lá 12 và −3.

Ở nút MIN phải, nút MAX đầu tiên trả về $\max(-4, -2) = -2$. Khi đó $\beta = -2 \leq \alpha = 1$, nên cắt hai nút MAX còn lại, chứa các lá (−2, 10) và (−5, −7).

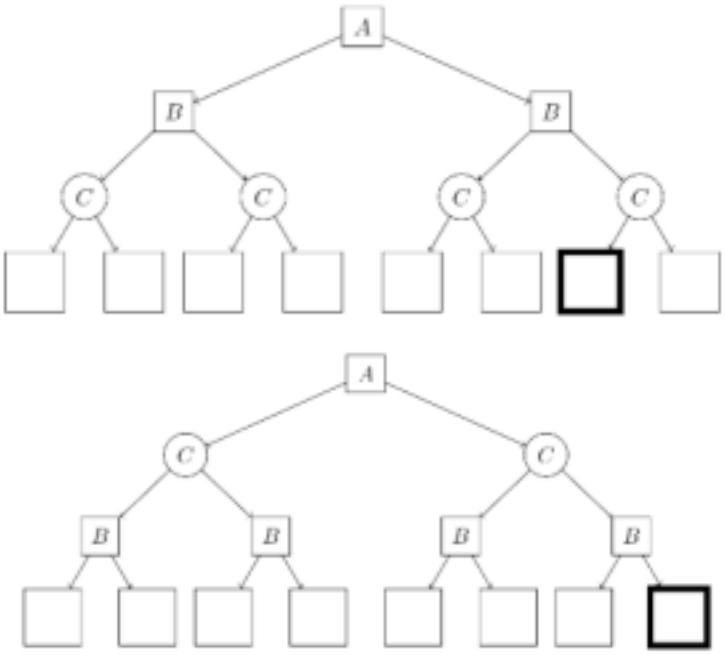
d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.

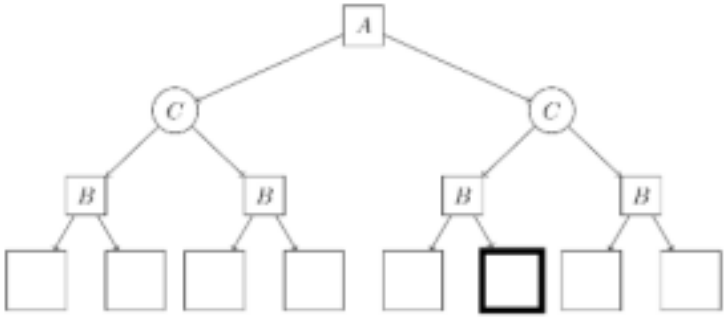
Các lá bị cắt là: 6; 12 và −3; −2 và 10; −5 và −7.

=> Tổng cộng 7 lá không cần thăm. Minimax đánh giá 18 lá; Alpha-Beta đánh giá 11 lá. Cả hai vẫn cho giá trị tại gốc là 1

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.





Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Có thể tạo ví dụ cắt tỉa nhờ cận tại nút MIN A. Đặt hai lá của nút C đầu tiên bên trái là 0 và 0, nên C có kỳ vọng 0 và A có $\beta = 0$. Ở nhánh B bên phải, đặt hai lá của nút C đầu tiên là 5 và 5, nên giá trị C = 5 và B hiện đã có giá trị ít nhất 5. Vì $5 > \beta = 0$, A sẽ không chọn nhánh phải; do đó phần cây con tiếp theo bên phải (bao gồm lá viền đậm trong hình) không cần xét. Điều kiện là giá trị đã xác định tại B lớn hơn cận β của A; không dùng trường hợp bằng nhau.

b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Không thể bảo đảm cắt lá viền đậm trong trường hợp tổng quát chỉ dựa trên thông tin đề cho. Lá đó thuộc cây có nút ngẫu nhiên C; kỳ vọng của C phụ thuộc các lá còn lại. Nếu không có cận trên/dưới đã biết cho những lá chưa xét thì các giá trị chưa biết có thể làm thay đổi kỳ vọng và quyết định của nút cha. Vì vậy không có cận hợp lệ để chứng minh việc cắt tỉa

c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Không thể bảo đảm cắt riêng lá viền đậm trong trường hợp tổng quát khi chưa biết cận của các lá chưa xét. Ở nút C, một lá chưa xét vẫn có thể làm thay đổi kỳ vọng; không thể truyền cận α–β qua nút ngẫu nhiên như qua nút MAX/MIN thông thường. Nếu đề bổ sung miền giá trị tiện ích hữu hạn đã biết, có thể tính cận của kỳ vọng để xét khả năng cắt tỉa; dữ kiện hiện tại chưa đủ

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

Ở cây Minimax, nút MAX/MIN có thể được cắt khi một cận đã đủ chứng minh các con còn lại không thể thay đổi quyết định. Với nút ngẫu nhiên, giá trị là trung bình có trọng số của các kết quả; một lá chưa xét vẫn có thể ảnh hưởng kỳ vọng. Muốn cắt tỉa cần biết cận hợp lệ của tiện ích chưa xét và tính được cận kỳ vọng, nếu không thì không thể kết luận an toàn

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có $V_M \leq V_E$. ☑ Đúng ☐ Sai

Tại mỗi nút bị thay, kỳ vọng có trọng số của các giá trị con luôn lớn hơn hoặc bằng giá trị nhỏ nhất trong các giá trị đó: $E[X] \geq \min(X)$. Các nút MAX bảo toàn quan hệ này khi lấy giá trị lớn nhất. Do đó tại gốc MAX $V_M \leq V_E$

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M. ☑ Đúng ☐ Sai

Với mọi nút trước đây là MIN, bất kỳ phân phối xác suất nào trên các con cũng có kỳ vọng không nhỏ hơn giá trị nhỏ nhất của các con. Do đó, nếu tiếp tục dùng hành động được chọn bởi chính sách Minimax, kết quả kỳ vọng của chính sách đó ít nhất bằng mức bảo đảm $V_{M.}$

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E. ☐ Đúng ☑ Sai

V_E là giá trị tối ưu theo Expectimax, tức tối đa hóa kỳ vọng. Chính sách Minimax tối ưu theo tiêu chí chống lại đối thủ bất lợi nhất, nên có thể chọn nhánh khác nhánh tối ưu theo kỳ vọng. Vì thế chính sách Minimax không được bảo đảm đạt V_E.

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

☐ Minimax với tiện ích trạng thái cuối +1/−1

☑ Expectimax với tiện ích trạng thái cuối +1/−1

☑ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua ☐ Không phương án nào

d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.

Expectimax: phù hợp khi các kết quả ngẫu nhiên và xác suất của chúng được mô hình hóa; chọn hành động có tiện ích kỳ vọng cao nhất.

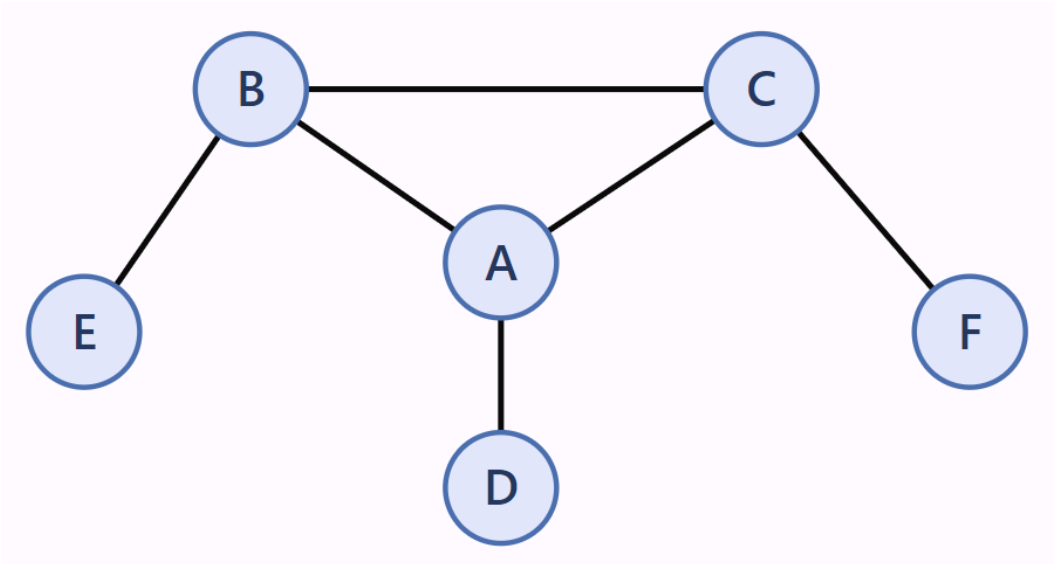
Value Iteration: phù hợp nếu biểu diễn được bài toán thành MDP với trạng thái, xác suất chuyển và phần thưởng. Có thể dùng phần thưởng 0 ở trạng thái trung gian và +1/−1 ở trạng thái kết thúc.

Minimax: không phù hợp nếu trò chơi không có đối thủ đối kháng cần tối ưu hóa theo trường hợp xấu nhất.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh)



b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Forward Checking chỉ lọc miền của các biến chưa gán có cạnh nối trực tiếp với A: B, C và D; E và F không kề trực tiếp với A nên không bị tác động trực tiếp ở bước này.

c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?

Các biến chưa gán kề trực tiếp với B là C và E. Vì vậy Forward Checking tại B có thể lọc miền của C và E. Miền C có thể đã được thu hẹp từ lần gán A trước đó.

d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?

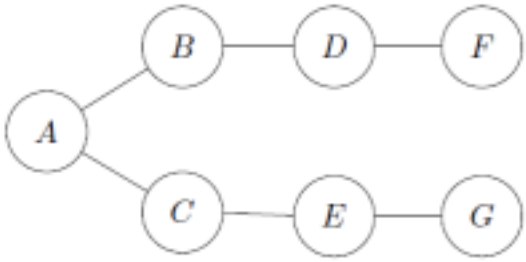
Trực tiếp, các miền B, C và D có thể thay đổi do ràng buộc với A. Nếu việc loại giá trị ở B hoặc C làm mất giá trị hỗ trợ, lan truyền có thể tiếp tục đến E qua B và F qua C. Do đó các miền có thể bị ảnh hưởng là B, C, D, E, F; miền nào thực sự thay đổi phụ thuộc miền giá trị và các quan hệ ràng buộc cụ thể

e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?

Việc gán B có thể làm thay đổi miền C và E. Nếu miền C thay đổi thì Arc Consistency có thể lan truyền tiếp sang F. Các biến D, E, F cũng có thể đã bị ảnh hưởng từ bước trước; không thể khẳng định miền nào chắc chắn thay đổi nếu chưa biết miền giá trị và bảng ràng buộc cụ thể.

Các ý f–h sử dụng đồ thị Hình 3 gồm bảy biến A–G với các cạnh A–B, A–C, B–D, D–F, C–E và E–G. AC được chạy ban đầu và sau mỗi hai phép gán. Chỉ biết kích thước miền d và đồ thị là chưa đủ để tính số backtrack chính xác; kết quả còn phụ thuộc quan hệ ràng buộc và giá trị nào bị loại khi lan truyền. Phân tích trường hợp xấu nhất dưới đây giả định AC có thể không loại giá trị nào cho đến khi mâu thuẫn xuất hiện.

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến (nguồn: rec3.pdf, trang 6)

f) Thứ tự A → B → C → D → E → F → G.

AC chạy ban đầu, sau A,B; sau C,D; sau E,F. Nếu AC không phát hiện mâu thuẫn sớm, tìm kiếm có thể phải thử các tổ hợp của bảy biến. Cận thô số tổ hợp hoàn chỉnh là d^7; số nút gán từng phần tối đa theo cây tìm kiếm là $1 + d + d^2 + \ldots + d^7 = (d^8 - 1)/(d - 1)$ với $d > 1$.

Đây là cận số nút gán/nhánh tìm kiếm, không phải số backtrack chính xác; số backtrack tùy vị trí mâu thuẫn và kết quả AC.

g) Thứ tự G → E → C → A → B → D → F.

AC chạy ban đầu, sau G,E; sau C,A; sau B,D. Trong trường hợp xấu nhất AC không thu hẹp miền trước khi mâu thuẫn xuất hiện, cận thô vẫn là d^7 tổ hợp hoàn chỉnh và $1 + d + \ldots + d^7$ nút gán từng phần. Thứ tự này có thể phát

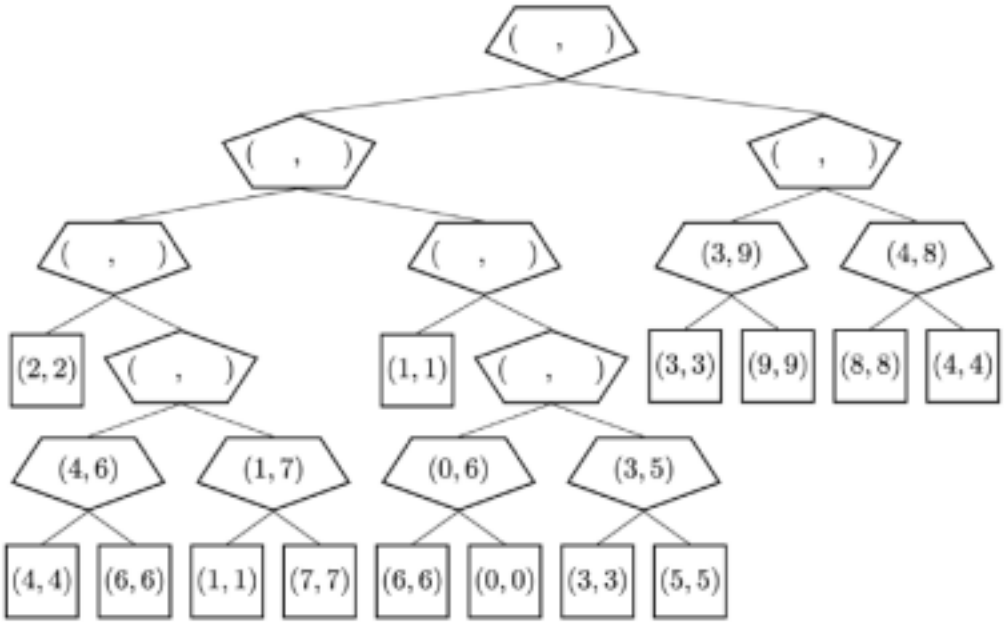
hiện mâu thuẫn sớm hơn hoặc muộn hơn tùy các ràng buộc; không thể tính số backtrack chính xác từ dữ kiện hiện có.

h) Thứ tự G → F → E → D → C → B → A.

AC chạy ban đầu, sau G,F; sau E,D; sau C,B. Trong trường hợp xấu nhất, nếu AC không loại giá trị sớm, cận thô cũng là d^7 tổ hợp hoàn chỉnh và $1 + d + \ldots + d^7$ nút gán từng phần. Muốn so sánh số backtrack thực tế giữa ba thứ tự cần có miền giá trị và các ràng buộc cụ thể, điều mà đề chưa cung cấp.

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Hình 4. Cây trò chơi Pacman–Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper

Nút Pacman phía trên (4,6) và (1,7) chọn u lớn hơn: max(4,1) = 4 => (4,6).

Nút Pacman phía trên (0,6) và (3,5) chọn u lớn hơn: max(0,3) = 3 => (3,5).

Nút Casper có hai lựa chọn (2,2) và (4,6): u = min(2,4) = 2; v = max(2,6) = 6 => (2,6).

Nút Casper có hai lựa chọn (1,1) và (3,5): u = min(1,3) = 1; v = max(1,5) = 5 => (1,5).

Nút Pacman phía trên (2,6) và (1,5) chọn nhánh có u = 2 => (2,6).

Nút Pacman phía trên (3,9) và (4,8) chọn nhánh có u = 4 => (4,8).

Nút gốc Casper so sánh (2,6) và (4,8): u = min(2,4) = 2; v = max(6,8) = 8.

Vậy nút gốc là (2,8).

* u là giá trị Pacman dự đoán khi tin Casper đối kháng; v là kết quả thực tế khi Casper thân thiện. Vì cách chọn ở Casper khác nhau theo hai thành phần, u và v có thể khác nhau.

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

Duyệt cây từ trái sang phải. Tại nút Pacman có hai nhánh dẫn đến các cặp ((2,6)) và ((1,5)), Pacman chọn nhánh có (u=2). Vì Casper ở nhánh còn lại có một nút con mang giá trị ((1,1)), giá trị (u) của Casper chắc chắn không lớn hơn 1. Do đó, nhánh còn lại không thể được Pacman lựa chọn, nên có thể bỏ qua phần cây con chưa duyệt bên dưới nhánh này.

Ở các nút Casper khác, không thể cắt tỉa chỉ dựa vào (u), vì Casper còn phải lấy giá trị (v) lớn nhất. Chẳng hạn, tại nút có hai lựa chọn ((2,2)) và ((4,6)), lựa chọn có (u=4) không làm thay đổi giá trị (u=2) của Casper, nhưng lại làm tăng (v) từ 2 lên 6. Vì vậy, vẫn phải xét nhánh đó để tính đúng cặp ((u,v)).

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Khi Pacman biết Casper thân thiện, Pacman không còn dùng u để dự đoán sai hành vi của Casper. Tại mọi nút Casper, chỉ cần tối đa hóa tiện ích thực tế v; tại nút Pacman cũng chọn nhánh có v lớn nhất. Do đó cây trở thành bài toán tối đa hóa tiện ích ở cả hai loại nút.

Tính từ dưới lên: các nút ngay trên lá lần lượt có giá trị max(4,6)=6; max(1,7)=7; max(6,0)=6; max(3,5)=5. Các nút Pacman/Casper phía trên tiếp tục chọn giá trị lớn nhất của các con. Ở hai nhánh phải có các cặp (3,9) và (4,8), tiện ích thực tế tốt hơn là 9 ở nhánh (3,9). Khi tất cả các nút chọn theo v, giá trị gốc bằng tiện ích lớn nhất có thể đạt được trong cây, tức 9; Pacman chọn đường dẫn tới lá có tiện ích 9.

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
Breadth-First Search (BFS)	Có nếu hệ số phân nhánh hữu hạn và lời giải ở độ sâu hữu hạn. Với đồ thị, cần tránh lặp/chu trình bằng tập visited.	Có nếu chi phí mỗi bước bằng nhau; nếu chi phí khác nhau thì không bảo đảm tìm đường có tổng chi phí nhỏ nhất.
Depth-First Search (DFS)	Không bảo đảm trong không gian vô hạn hoặc có nhánh vô hạn. Trong không gian hữu hạn, có thể đầy đủ nếu xử lý chu trình và không lặp vô hạn.	Không bảo đảm; có thể gặp lời giải sâu/đắt trước lời giải nông/rẻ hơn.
Iterative Deepening Search (IDS)	Có nếu hệ số phân nhánh hữu hạn và lời giải ở độ sâu hữu hạn.	Có khi chi phí mỗi bước bằng nhau, vì tìm lời giải ở độ sâu nhỏ nhất.
A* Search	Có nếu hệ số phân nhánh hữu hạn, chi phí mỗi bước có cận dưới dương và điều kiện heuristic/chi phí phù hợp.	Tree search: heuristic admissible là điều kiện chuẩn để bảo đảm tối ưu. Graph search: heuristic consistent; nếu không consistent, cần mở lại nút khi tìm được đường tốt hơn.

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận

BFS và IDS tìm lời giải có số bước ít nhất, không nhất thiết có tổng chi phí thấp nhất. Vì thế, chúng chỉ tối ưu khi chi phí mỗi bước bằng nhau. A* đánh giá nút bằng f(n)=g(n)+h(n), trong đó g(n) là chi phí đã đi và h(n) là ước lượng chi phí còn lại. Heuristic admissible không được ước lượng quá chi phí thật; heuristic consistent còn bảo đảm f không giảm dọc đường đi. Các điều kiện này giúp A* giữ được tính tối ưu. Nếu điều kiện không thỏa, A* có thể trả về lời giải không tối ưu.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a) Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Chi phí của đường tối ưu cũ tăng n, nhưng đường đó có thể không còn tối ưu. Cần cập nhật chi phí X→Y, tính lại g(Y) và các nút hậu duệ có đường đi phụ thuộc cạnh này; đồng thời xét các đường thay thế. Chi phí tối ưu mới có thể tăng, hoặc không đổi nếu còn đường khác có chi phí bằng mức tối ưu cũ. Không thể khẳng định nó tăng đúng n trong mọi trường hợp.

b) Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Chi phí của đường cũ giảm n. Cập nhật g(Y) và lan truyền mức cải thiện tới các nút hậu duệ. Đường cũ vẫn là ứng viên rất tốt; trong mô hình chỉ thay đổi cạnh này, chi phí của đường đó giảm n. Cần duy trì/kiểm tra các giá trị đường đi để xác nhận lời giải tối ưu mới.

c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Đường tối ưu cũ không dùng cạnh này nên chi phí của nó không đổi. Việc làm một cạnh khác đắt hơn không thể tạo ra một đường rẻ hơn đường tối ưu cũ; do đó chi phí tối ưu vẫn giữ nguyên. Cập nhật cạnh để các lần mở rộng sau dùng đúng chi phí mới.

d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Cạnh rẻ hơn có thể tạo ra đường mới tốt hơn. Cập nhật c(X,Y), tính ứng viên g(Y)=g(X)+c(X,Y), rồi lan truyền các giá trị g tốt hơn tới hậu duệ và tiếp tục mở rộng hàng đợi. Nếu đường mới đến đích rẻ hơn thì cập nhật lời giải; nếu không, đường tối ưu cũ vẫn giữ nguyên.

e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Đường tối ưu cũ vẫn hợp lệ và có chi phí không đổi. Nếu giả thiết ban đầu rằng A* đã xác định tối ưu trên đồ thị đầy đủ vẫn được giữ, tăng chi phí một cạnh không nằm trên đường tối ưu không thể làm giảm chi phí tối ưu; kết quả tối ưu vẫn giữ nguyên. Trong triển khai tìm kiếm tăng dần, cập nhật chi phí khi cạnh được phát hiện.

f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Cạnh mới rẻ hơn có thể tạo ra đường tốt hơn nhưng đường đó chưa được xét. Chưa thể kết luận chi phí tối ưu có thay đổi hay không. Khi phát hiện cạnh, cập nhật chi phí và các đường đi liên quan, mở lại/cập nhật những nút có g tốt hơn, rồi tiếp tục tìm kiếm cho đến khi điều kiện kết thúc của A* được thỏa mãn.

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

• A: act(percept) → f_A()

• B: act(percept) → f_B(percept)

• C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

Các lớp có thể hành động hợp lý hoàn hảo: B và C. Khi quan sát đầy đủ, percept hiện tại cho biết trạng thái cần thiết để chọn hành động tối ưu trong môi trường xác định. C cũng làm được vì có thể sử dụng percept hiện tại và bỏ qua phần lịch sử không cần thiết. A không thể bảo đảm trong mọi môi trường thuộc lớp này vì không phản ứng theo percept; nếu trạng thái khác nhau đòi hỏi hành động khác nhau, một hàm cố định sẽ không luôn đúng.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

C là lớp phù hợp nhất vì có thể dùng lịch sử percepts để suy luận trạng thái hiện tại trong môi trường xác định. B chỉ đủ nếu percept hiện tại tự nó chứa đủ thông tin để chọn hành động tối ưu; điều này không được bảo đảm với mọi môi trường quan sát một phần. A không thể bảo đảm vì bỏ qua thông tin quan sát.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

C là lớp phù hợp nhất vì có thể tổng hợp lịch sử quan sát để duy trì niềm tin về trạng thái và chọn hành động có tiện ích kỳ vọng cao nhất. Kết luận này giả định tác tử biết hoặc có thể học mô hình xác suất và phần thưởng. B chỉ đủ trong trường hợp đặc biệt khi percept hiện tại đã đủ để xác định hành động tối ưu; A không bảo đảm được.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.

Đúng. Ví dụ, xét môi trường chỉ có một hành động khả dụng ở mọi trạng thái; khi đó mọi tác tử đều phải thực hiện hành động duy nhất và hành động đó là tối ưu. Một ví dụ khác là môi trường mà mọi hành động đều dẫn đến cùng kết quả và cùng tiện ích. Trong các môi trường như vậy, ngay cả tác tử A không phụ thuộc percept cũng hành động hợp lý.

— Hết —